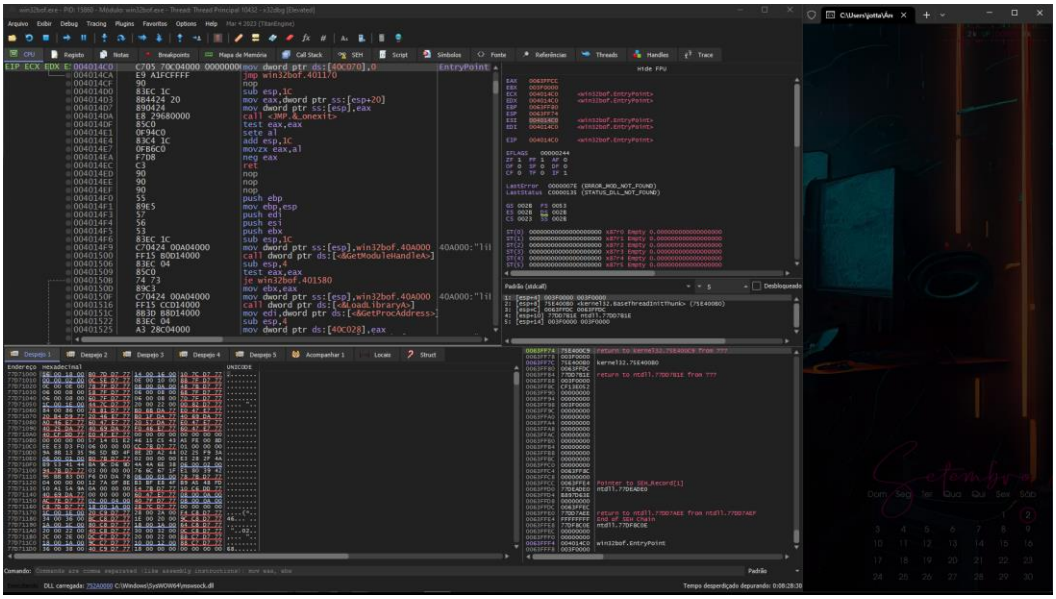
Skills Assessment

Write-up por: João Pedro T. A. T. Parisi

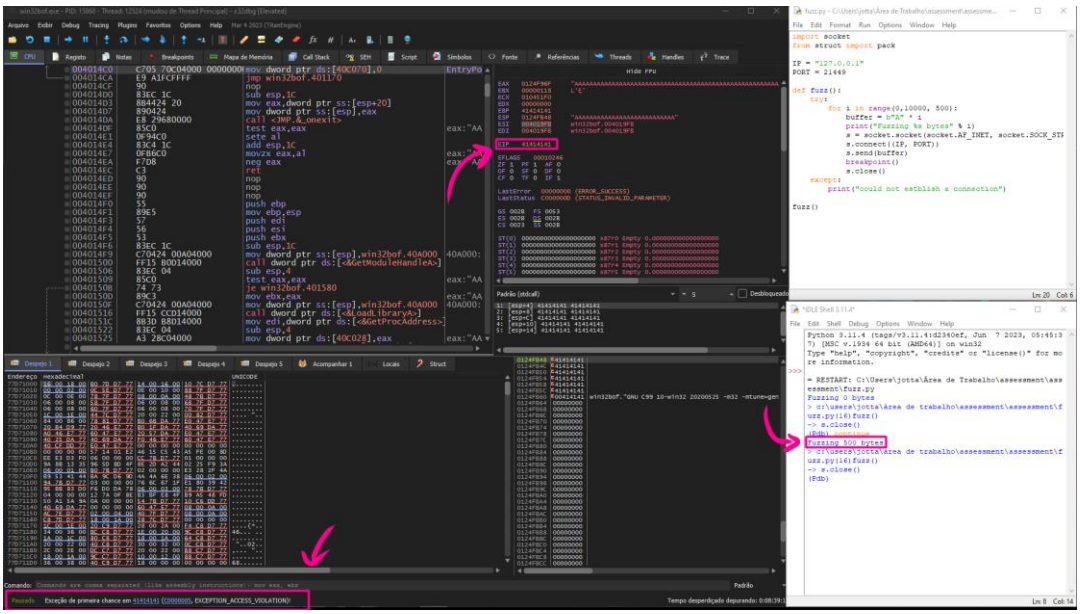
Comecei abrindo o aplicativo, no xdbg na minha máquina e comecei a seguir os passos para identificação da vulnerabilidade de buffer overflow…



1. **Fuzzing Parameters:**

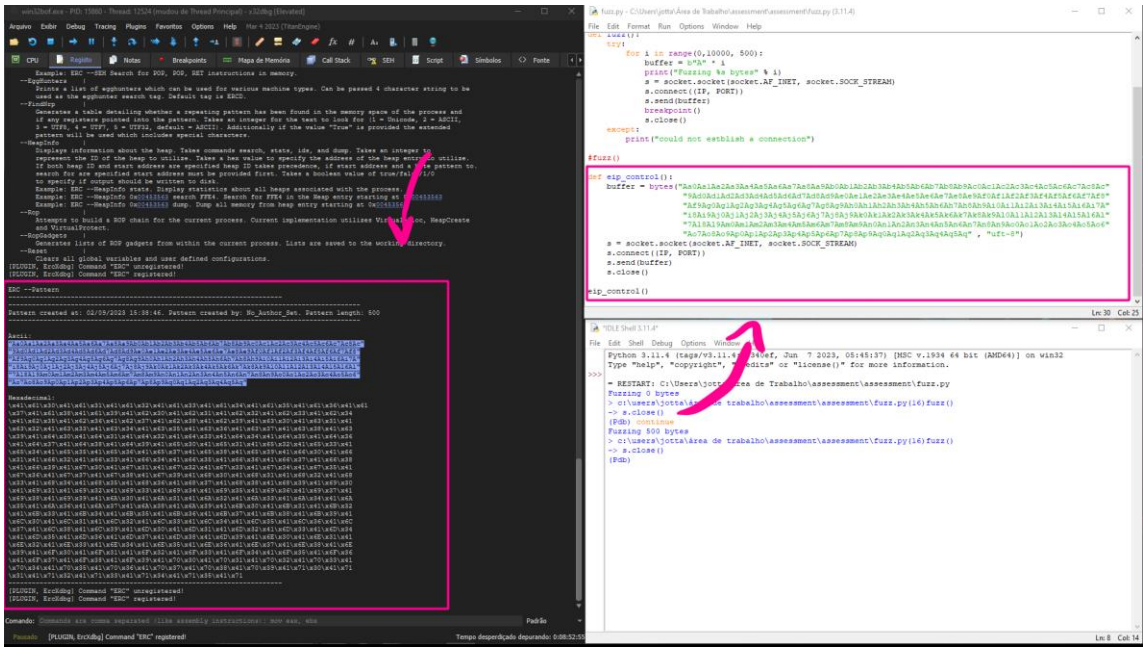
Comecei realizando o fuzzing dos parâmetros de modo remoto para contatar a aplicação e ver se ela quebra.

Criei um script em python para enviar até 10000 caracteres “A”, começando do 0 e pulando de 500 em 500, mostrando cada uma das iterações no Python Debugger. O resultado foi o crash do programa sobrescrevendo o EIP logo nos primeiros 500 caracteres…

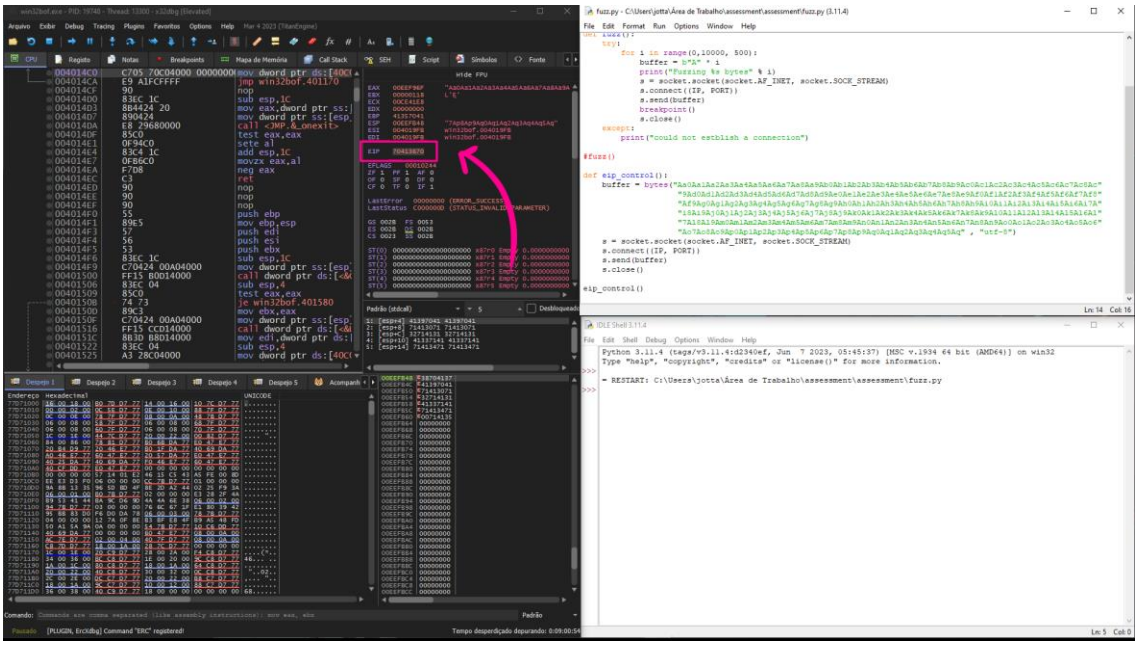


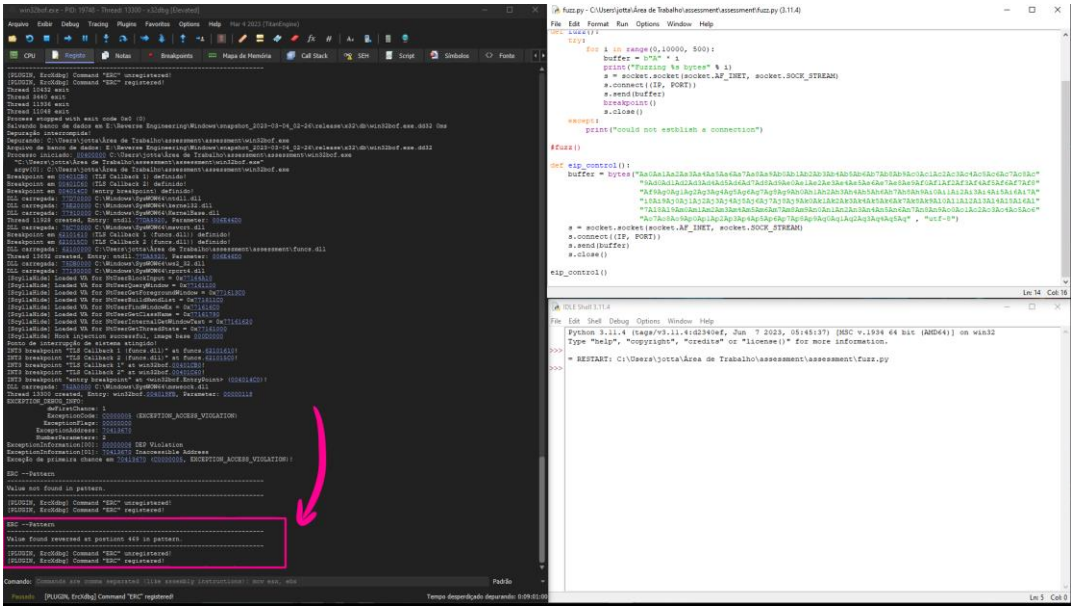
2. Controlling EIP:

Para identificar o deslocamento do EIP criei uma sequência única com o plugin ERC de 500 bytes e a enviei através do script python:

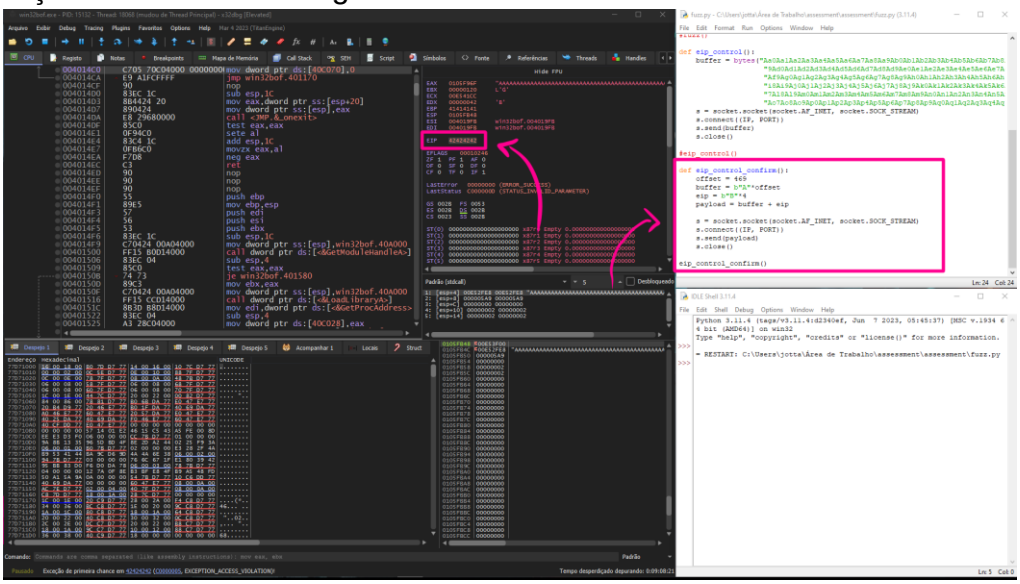


Rodando o script podemos notar a entrada dele no EIP e podemos usar o comando o ERC para identificar sua posição baseado em nossa sequência:



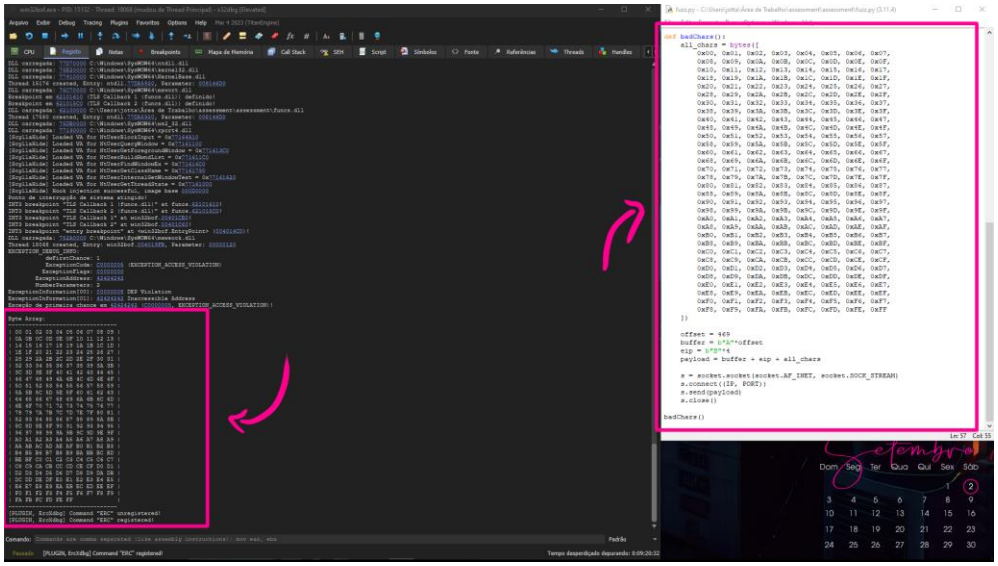


O resultado é 469 caracteres até o input sobrescrever o EIP como visto acima… e a confirmação do controle do registrador:

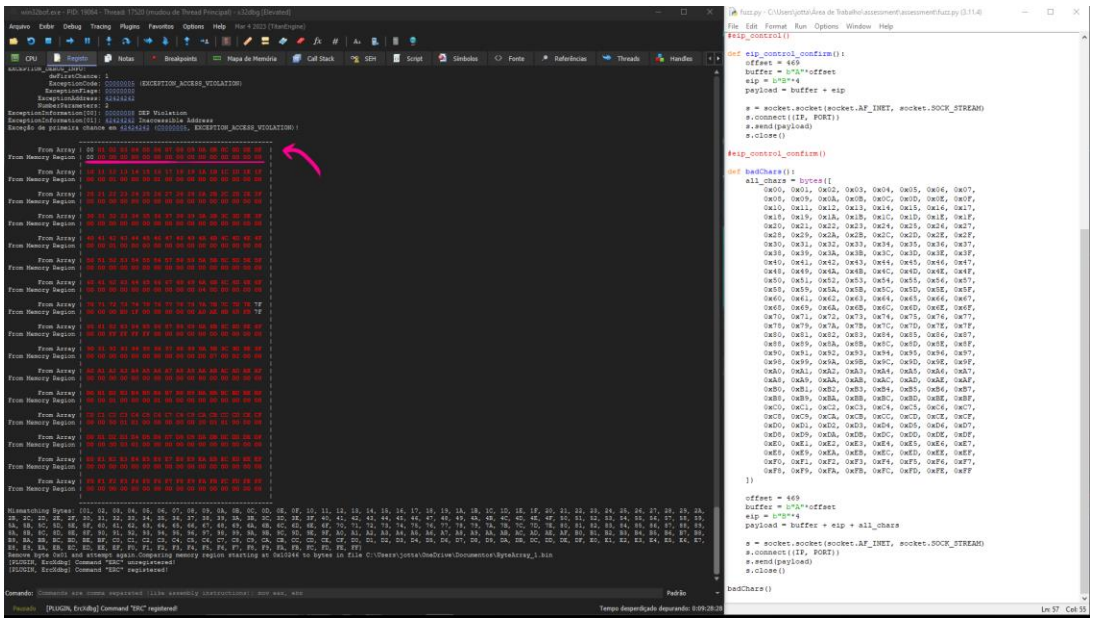


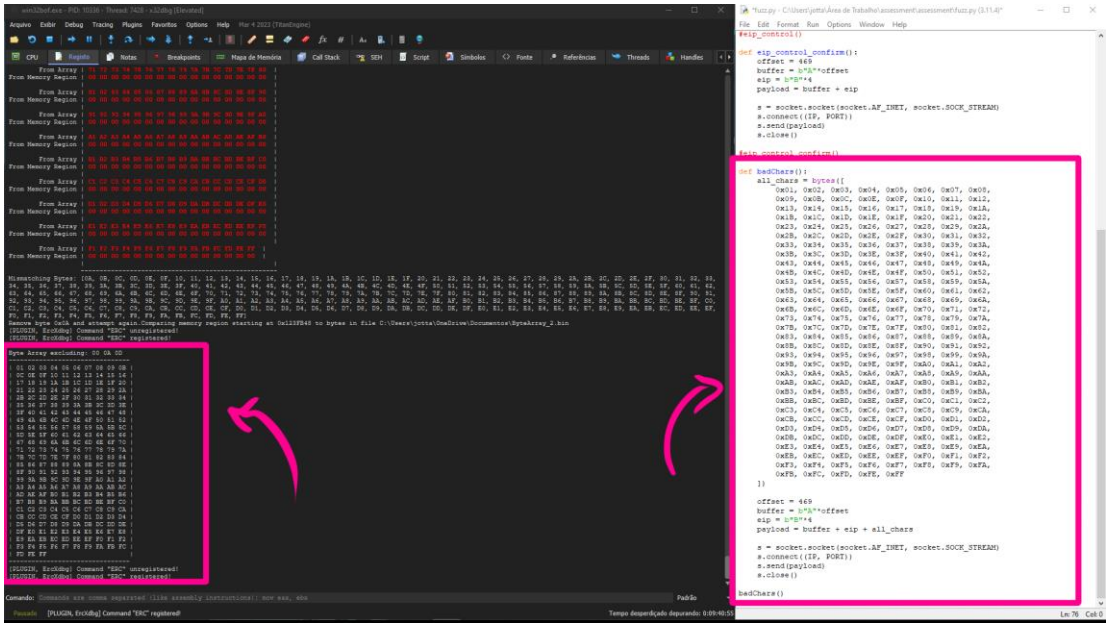
3. Identifying Bad Characters:

Fiz a identificação dos caracteres ruins, que possam indicar ao input que a string enviada foi terminada… Para isso criei um bytearray com todos o caracteres possíveis no ERC e adicionei ao meu script:

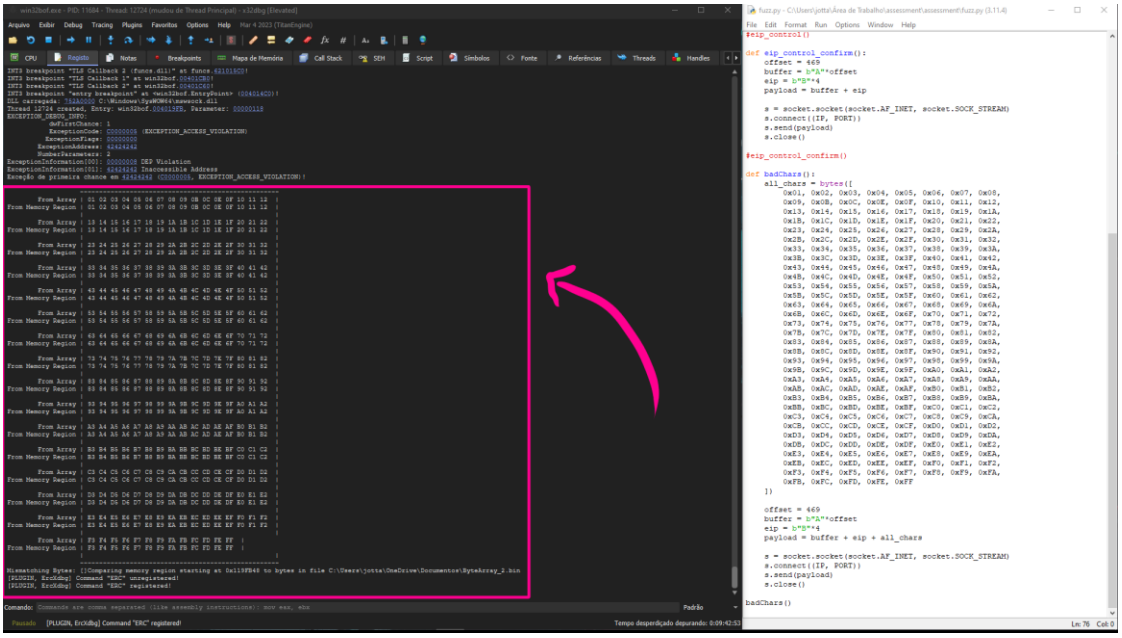


Rodando podemos fazer a análise da pilha com o ERC e ver quais caracteres estão faltando, no caso como notei que o 00 está terminando o array já vou gerar o próximo com os caracteres ruins mais comuns(0x00, 0x0a, 0x0d):



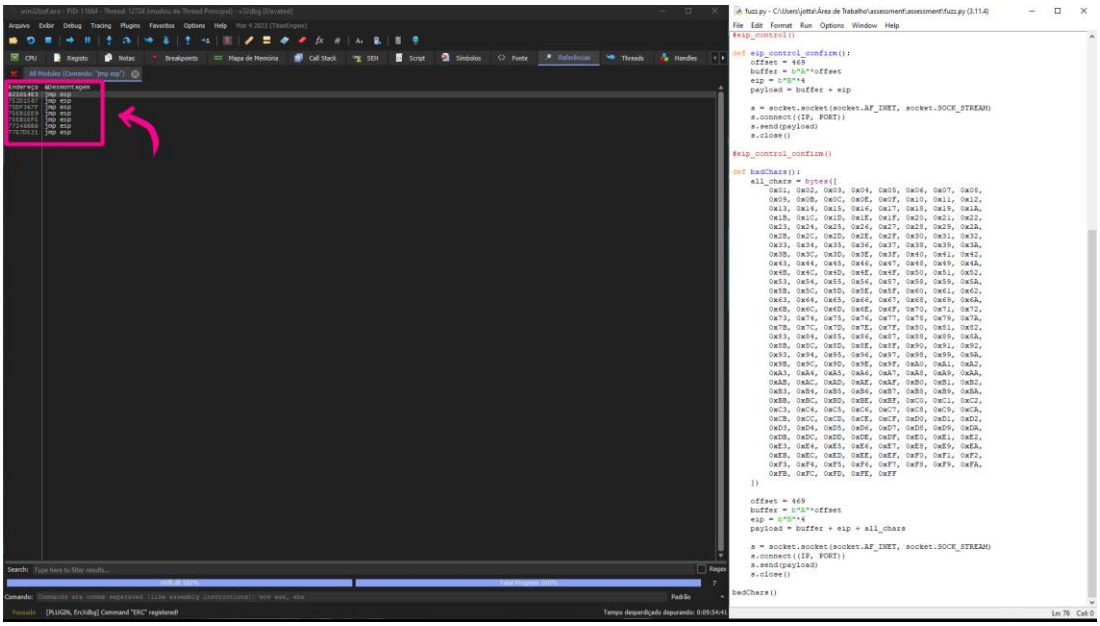


Rodando sem os caracteres ruins citados anteriormente podemos ver que não possuímos mais caracteres ruins:

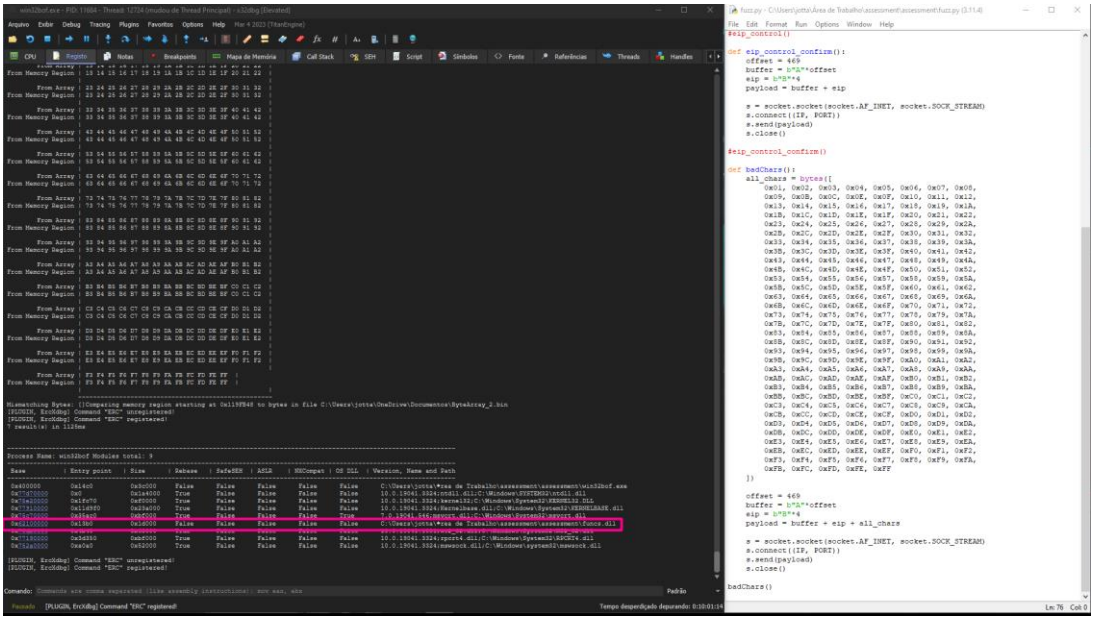


4. Finding a Return Instruction:

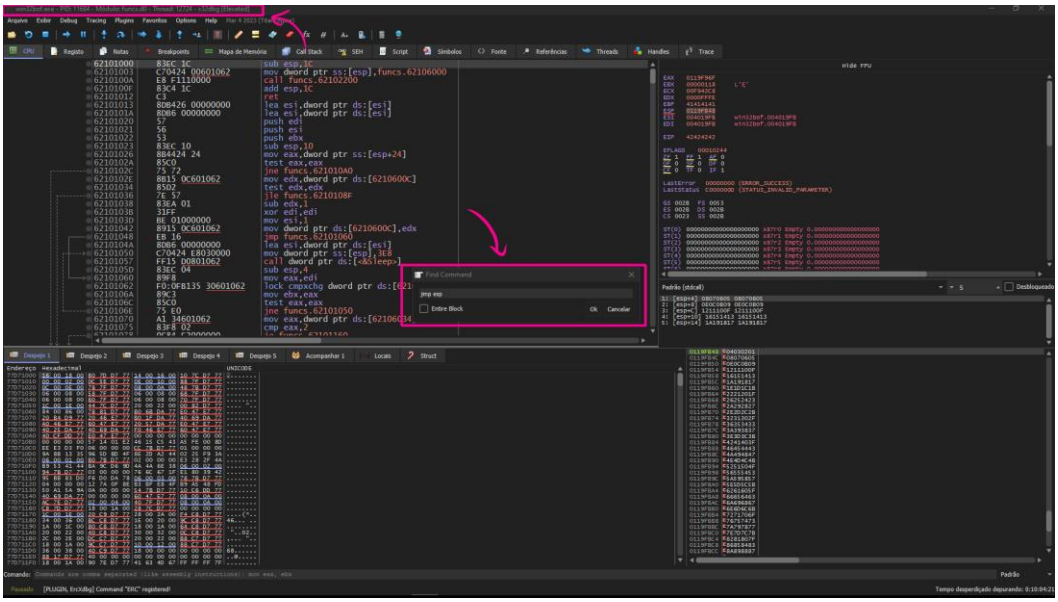
Procurei no xdbg por instruções que pulem pro registrador esp(ponteiro de pilha para a pilha do sistema)

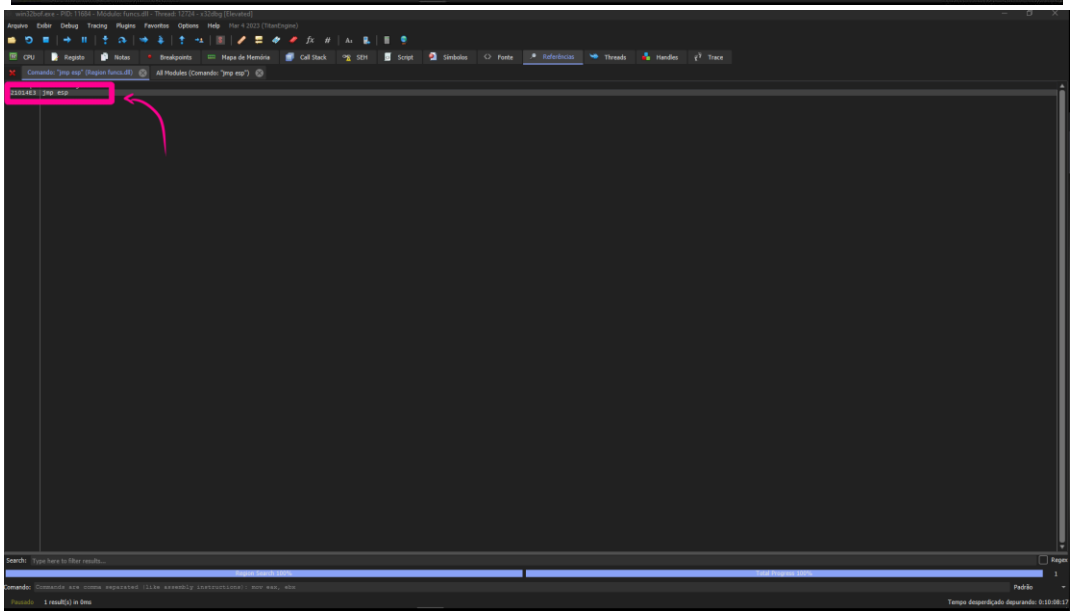


Depois usei o ERC para ver os arquivos carregados pelo programa na pilha, entre os arquivos vou utilizar do funcs.dll do próprio programa pois por não ser do SO, ele não fica em uma região de memória protegida:



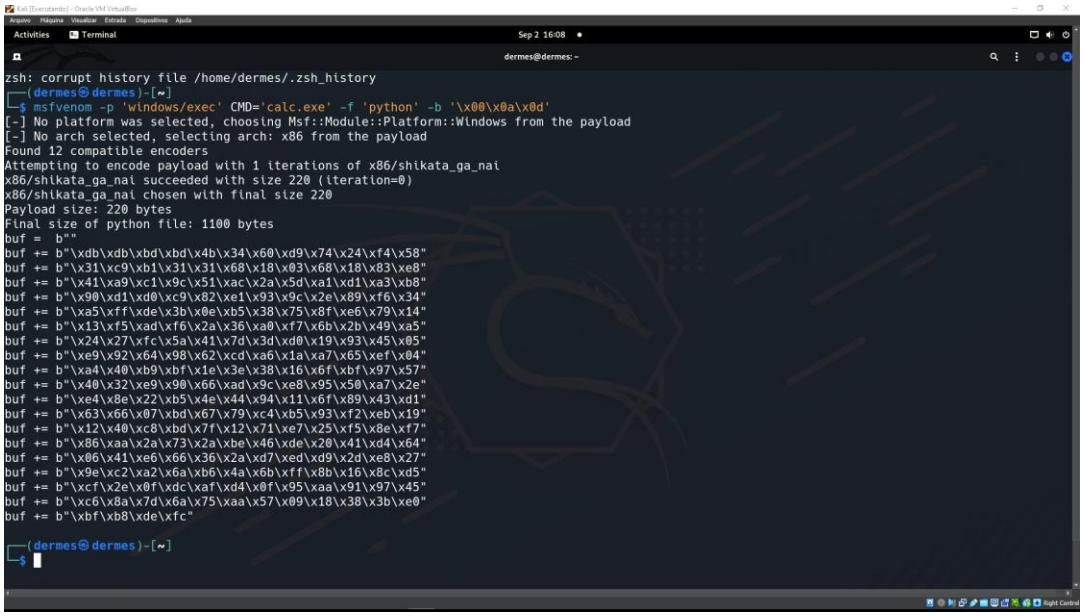
Carregando o funcs.dll no xdbg e pulando para a instrução “jmp esp” eu acho o endereço de memória da instrução:



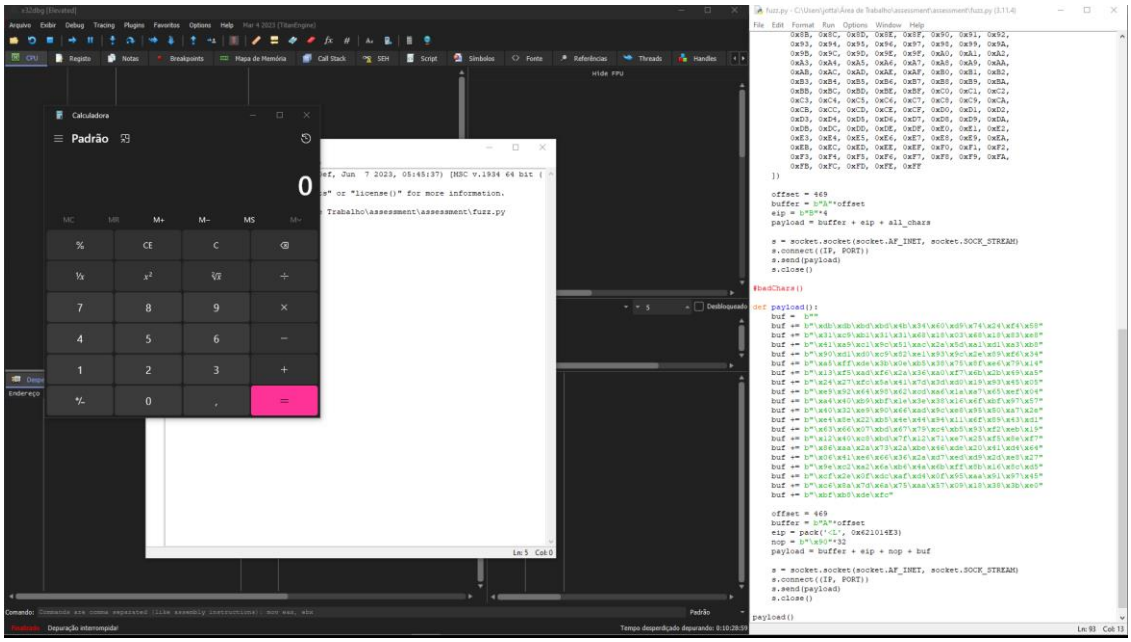


5. Jumping to Shellcode:

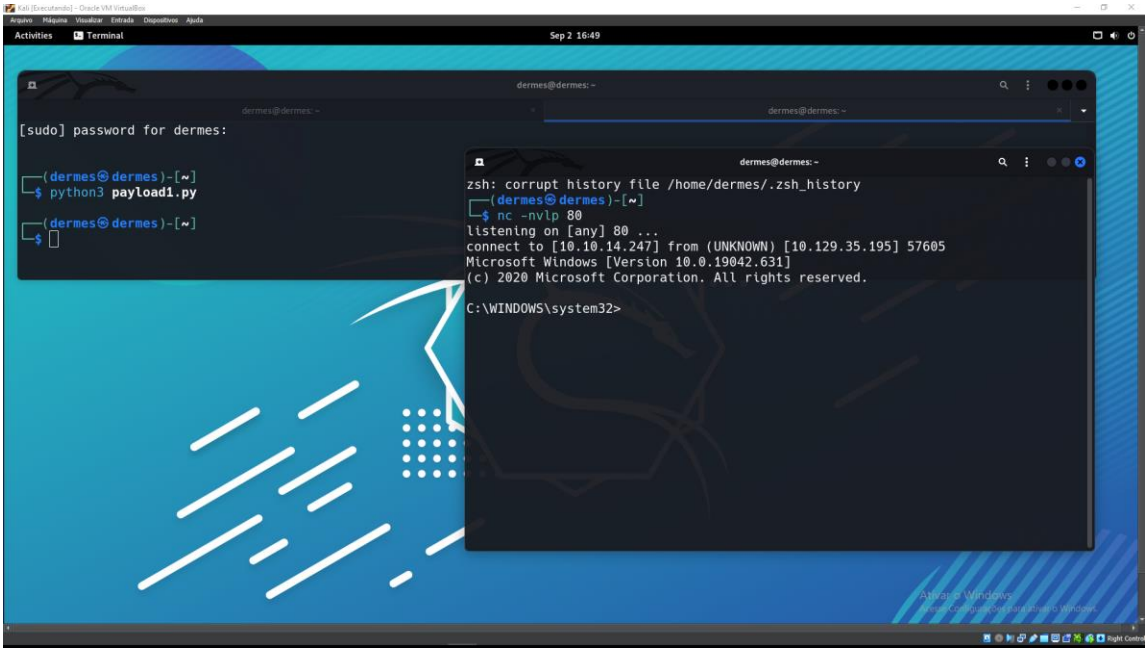
Eu subi uma VM do kali linux, para gerar o shellcode com o msfvenom



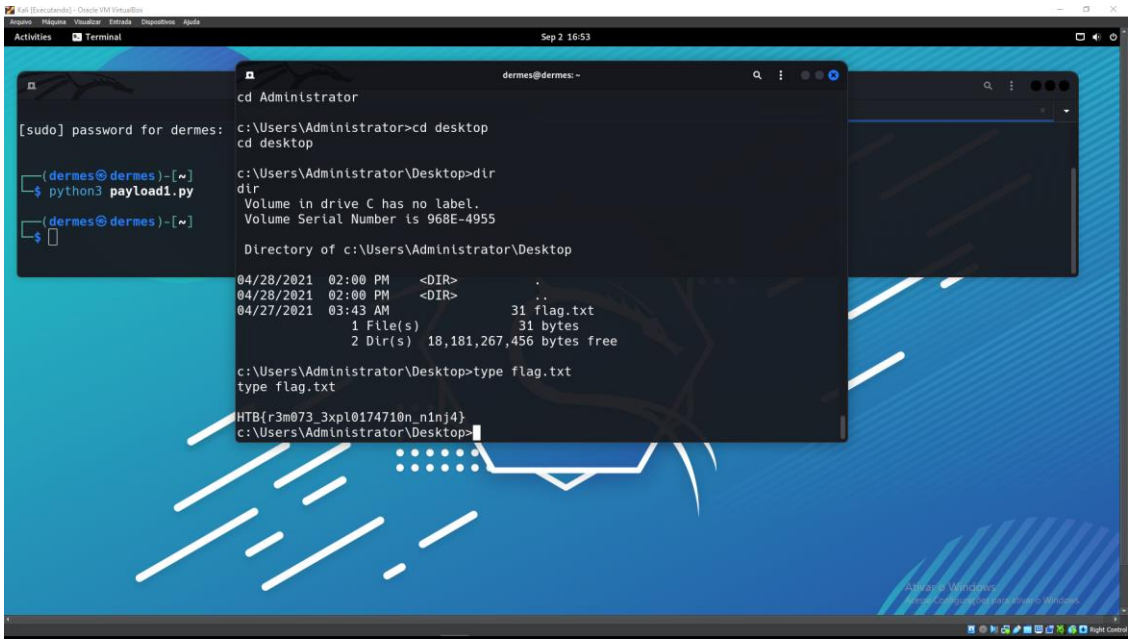
Colei o payload no meu script python para confirmar se o payload funciona e executei, e aí o resultado:



Por fim, voltei ao kali para gerar um payload de shell reverso e conectar remotamente ao alvo… Colocando o payload no script e usando o netcat para me conectar, pude obter acesso a máquina do alvo:



Desta forma, navegando até a área de trabalho do administrador do alvo pude obter a flag para concluir o exercício… Desafio concluído com sucesso!!
Flag: HTB{r3m073_3xpl0174710n_n1nj4}



Stack-Based Buffer Overflows o... | Documento sem título - Docum...

academy.hackthebox.com/module/details/89

Módulos do curso... | YouTube | CTFtime.org / Real... | Welcome [Root Me... | Hack The Box: Hack... | picoCTF - Sign Up | TryHackMe | Red Te... | diagrams.net | Course: Information... | Certificação A+ (Pla... | STR 6.0 -AULA 01 -... | Placa-mãe Asus Tuf... | 5 maneiras infalíveis...

HTB ACADEMY | Search Academy | Purchase Cubes | Tricta

Tricta
Student
100

LEARN
- Dashboard
- Exams
- Modules
- Paths
- Academy x HTB Labs

MY ACHIEVEMENTS
- My Certificates
- My Badges 5

REFERRALS
- Invite friends

GET HELP
- Help Center
- FAQ

STACK-BASED BUFFER OVERFLOWS ON WINDOWS X86

Module / Details

View

Stack-Based Buffer Overflows on Windows x86

Tier 0 | Medium | Offensive | 8 hours

This module is your first step into Windows Binary Exploitation, and it will teach you how to exploit local and remote buffer overflow vulnerabilities on Windows machines.

★★★★★

Created by 21y4d

Module Summary

Windows binary exploitation has advanced throughout the years, from basic stack overflow techniques to advanced security bypass techniques and heap exploitation. To reach a level of understanding that enables us to successfully exploit even the most advanced applications, we must have a firm grasp of the fundamentals of Windows binary exploitation, which is the main aim of this module.

The `Stack-Based Buffer Overflows on Windows x86` module is your first step in Windows `Binary Exploitation`, and it will take you through the following:

1. What is binary exploitation and buffer overflows
2. How to debug Windows programs
3. Basics of local and remote fuzzing of Windows programs
4. Finding and using return instructions to subvert the program execution flow
5. Crafting malicious payloads and scripts to gain local and remote control through buffer overflow vulnerabilities
6. Developing a functional multi-tier Python exploit for stack-based buffer overflows, which can be used as a basis for other buffer overflow exercises

Throughout the module, we will be attacking two different programs. First, we will generate a malicious `.wav` file that exploits a buffer overflow vulnerability in an audio converter to perform local privilege escalation. Then, we will move towards remote exploitation by attacking a remote server to gain remote code execution over it after debugging the vulnerable binary locally and developing an exploit.

In addition to teaching the above topics, this module will also cover:

Ativar o Windows
Acesse Configurações para ativar o Windows.